Data Engineering Capstone Project

DATA ENGINEERING CAPSTONE PROJECT

End-to-End Multi-Region Retail Analytics Platform

A portfolio-grade project designed to demonstrate production-level data engineering skills across ingestion, storage, transformation, orchestration, quality, deployment, and observability.

Scenario

You have been hired as a Data Engineer at **NorthStar Retail**, a fictional mid-size e-commerce company operating across multiple regions. The company sells physical goods online and wants a modern data platform that can support both **operational analytics** (near-real-time dashboards for ops teams) and **strategic analytics** (historical trend analysis, forecasting, and BI for leadership).

Today, NorthStar's data lives in scattered CSV exports, a single transactional Postgres database, and an unreliable nightly cron job. Leadership has asked you to design and build a platform from the ground up that can:

- Ingest both **historical/batch** data and **real-time/streaming** events
- Store data reliably and cost-efficiently at scale
- Transform raw data into clean, trusted, analysis-ready tables
- Run reliably on a schedule, with visibility into failures
- Prove its own correctness through automated data quality checks
- Be deployed and reproduced by anyone on the team (or in an interview) with minimal manual steps
- Be monitored, secured, and documented like a real production system

This project progressively builds that platform, one layer at a time. Each step produces a working, demonstrable artifact you can showcase.

Suggested datasets

Pick **one** primary dataset to anchor the project (or combine two if you want to demonstrate multi-source integration):

- **Olist Brazilian E-Commerce Dataset** (Kaggle) — orders, customers, payments, reviews, geolocation. Good relational structure for warehouse modeling.
- **NYC TLC Trip Record Data** — large, well-known, good for partitioning/scaling practice.
- **Instacart Market Basket dataset** — good for behavioral/event-style modeling.
- A **synthetic streaming event generator** you write yourself (recommended in addition to any of the above) to simulate live "clickstream" or "order placed" events — this is what makes the streaming portion possible without a live production system.

Note: Groups of 1–2 people may scope the streaming layer down to a single event type (e.g., only “order_placed” events) rather than a full clickstream. This scoping choice should be justified in the report.

Tasks

Step 1 — Architecture & System Design

Before writing code, design the system.

- Produce an **architecture diagram** showing all layers: sources → ingestion → storage → transformation → orchestration → serving/BI. Use a proper tool (draw.io, Excalidraw, Lucidchart).
- Write a short **design document** explaining your technology choices for each layer (e.g., why Airflow over Prefect, why a lakehouse over a plain warehouse, why Kafka or a lighter alternative for the streaming layer) and the trade-offs you considered.
- Define your **medallion architecture**: what constitutes Bronze (raw), Silver (cleaned/conformed), and Gold (business-ready aggregates) in your specific dataset.
- State your **non-functional requirements** explicitly: expected data volume, freshness SLAs (e.g., “gold tables refresh within 30 minutes of new data”), and what happens on failure.

Step 2 — Ingestion Layer

Build both a batch and a streaming ingestion path.

- **Batch**: Write a script/pipeline that pulls the historical dataset into raw storage (object storage such as S3/MinIO/GCS, or local equivalent), partitioned appropriately (e.g., by date). It must be idempotent — re-running it should not duplicate or corrupt data.
- **Streaming**: Write a lightweight event producer (Python script is fine) that emits synthetic events at intervals, and a consumer that lands them into raw storage or a message broker (Kafka, Redpanda, or a managed alternative like Kinesis — a simple queue-based simulation is acceptable at smaller scale).
- Handle **schema drift** gracefully: demonstrate what happens when an upstream field is added, renamed, or missing, and how your ingestion layer responds (fail loudly vs. adapt).
- Comment on your **idempotency and replay strategy** — if a pipeline fails halfway, how do you safely resume or reprocess?

Step 3 — Storage & Data Modeling

- Set up your Bronze/Silver/Gold storage layers (object storage + a queryable warehouse layer — DuckDB, Postgres, Snowflake trial, or BigQuery sandbox are all acceptable).
- Design a **dimensional model** (star or snowflake schema) for the Gold layer: identify fact tables and dimension tables, grain of each fact table, and slowly changing dimension (SCD) strategy for at least one dimension (e.g., customer address changes over time — implement SCD Type 2).
- Justify your **partitioning and file format** choices (Parquet vs. CSV, partition keys, compaction strategy) with reference to expected query patterns.

Step 4 — Transformation Layer

- Build your Bronze → Silver → Gold transformations using a modern transformation tool (**dbt** is strongly recommended; Spark/PySpark is a good alternative if you want to demonstrate distributed processing).
- Implement **at least one non-trivial transformation** beyond simple cleaning: a window-function-based metric (e.g., 7-day rolling revenue), a deduplication strategy, or a late-arriving-data handling mechanism.
- Document your **data lineage**: which Gold tables depend on which Silver tables, and how a consumer could trace a number in a dashboard back to its raw source.
- Version-control all transformation logic and explain your **environment strategy** (dev/staging/prod, or dbt targets).

Step 5 — Orchestration

- Orchestrate the full pipeline (ingestion → transformation → quality checks) using a workflow orchestrator (**Airflow**, Prefect, or Dagster).
- Your DAG(s) must include: explicit task dependencies, retry logic with backoff, alerting on failure (even a simple email/Slack webhook), and a documented **backfill procedure** for reprocessing a historical date range.
- Demonstrate that the pipeline is **scheduled**, not just manually triggered, and show a successful run history/screenshot.

Step 6 — Data Quality & Testing

- Implement automated data quality checks (dbt tests, Great Expectations, or custom assertions) covering at minimum: null checks on critical fields, referential integrity between fact and dimension tables, uniqueness of primary keys, and a business-logic check specific to your domain (e.g., “order revenue should never be negative”).
- Add at least one **unit test** for a piece of custom transformation logic (not just data-level tests).
- Produce a **data quality report** (even a simple generated summary) showing pass/fail status, and describe what your pipeline does when a check fails — does it halt downstream steps, quarantine bad rows, or alert only?

Step 7 — Deployment, Infrastructure & Observability

- Containerize the platform components with **Docker** (and docker-compose for local orchestration of the full stack) so it can be spun up with a small number of commands.
- If comfortable, provision infrastructure with **Terraform** (or document how you would, if using only local/free-tier resources).
- Set up basic **monitoring and observability**: pipeline run metrics, data freshness metrics, and at least one dashboard (Grafana, or even a simple generated HTML/BI dashboard) showing system health, not just business metrics.
- Address **security and governance** at a basic level: how are credentials/secrets managed (never hardcoded), is any PII in the dataset masked or access-controlled, and is there a simple data catalog or table-level documentation (dbt docs is sufficient).

Step 8 — Serving Layer & Final Showcase

- Expose the Gold layer to a consumption tool: a BI dashboard (Looker Studio, Metabase, Superset, or even a well-built notebook/Streamlit app) answering 3–5 real business questions your fictional stakeholders would ask (e.g., “which region has the highest month-over-month

revenue growth?”).

- Write a short **stakeholder-facing summary** (non-technical) of what the platform enables and what decisions it supports — this is your “explain it to your boss” deliverable, separate from the technical writeup.
- Record a **3–5 minute demo video or GIF walkthrough** of the running system — this is one of the most valuable portfolio artifacts you will produce.

Submission Requirements and Format

To make this usable as a **portfolio piece**, package it the way a hiring manager or reviewer would expect:

- **1 public GitHub repository** containing:
  - All source code (ingestion, transformation, orchestration DAGs, tests, IaC if used), organized into clear folders
  - A `README.md` that includes: project overview, architecture diagram, tech stack, setup/run instructions, and a link to the demo video
  - A `docs/` folder with the design document, data model (ER diagram), and data quality report
- **1 architecture diagram** (image, embedded in the README)
- **1 short demo video/GIF** (linked or embedded)
- **1 written report** (PDF or Markdown, excluding code) containing the answers to the open-ended discussion questions below — written as if presenting to a non-technical stakeholder *and* a technical peer reviewer, in clearly separated sections
- Optional but recommended: a short **blog-style write-up** (e.g., on Dev.to, Medium, or personal site) summarizing the project — this is often what recruiters actually read

Discussion questions to address in the written report:

- What were the main engineering trade-offs you made (cost vs. latency, simplicity vs. scalability), and why?
- Where would this architecture break first if data volume grew 100x, and how would you redesign that component?
- What would you do differently if this were a real production system with real users depending on it?
- How does the streaming layer’s design differ from the batch layer’s, and what did that teach you about the two paradigms?

Rubric

Category	Points	What it evaluates
System Design & Architecture	20	Clear, justified architecture diagram; sound layer separation (Bronze/Silver/Gold); realistic non-functional requirements; trade-offs explicitly reasoned about, not just asserted.
Technical Implementation	35	Working batch and streaming ingestion;

		correct dimensional modeling with at least one SCD implementation; non-trivial transformation logic; orchestrated, scheduled, retry-capable pipeline.
Data Quality & Reliability	15	Automated tests covering nulls, uniqueness, referential integrity, and a business rule; at least one unit test; documented failure-handling behavior.
Deployment & Observability	15	Reproducible via Docker/compose with minimal setup steps; basic monitoring/alerting in place; secrets handled properly; basic data governance/catalog present.
Communication & Portfolio Readiness	15	README a stranger could follow to run the project; architecture diagram and demo video present; written report clearly separates technical detail from stakeholder-facing explanation; no significant errors or unexplained jargon in the non-technical sections.

Total: 100 points

Tip: the single highest-leverage thing you can do for your portfolio is make the README good enough that someone can clone the repo and have it running in under 10 minutes. Most candidate projects fail on this, not on the technical content.